TASK 11 – Recommend and Defend the Final Design

1. Recommended Final Design

I recommend an object-oriented, integer-paise-based fare system using a SmartCard class, named constants and a separate fare-calculation function.

Component	Recommendation	Reason
Money type	long long (paise)	Avoids floating-point rounding and supports large network totals.
Time type	int (hour)	Simple and suitable for checking peak-hour boundaries.
Zone type	enum class Zone	Prevents invalid zone values from being used accidentally.
Fare constants	Named constants	Makes fare rules easy to locate and update.
Smart card	SmartCard class	Keeps balance private and controls recharge and deduction.
Fare calculation	Separate function	Keeps fare rules independent from card operations.

The SmartCard class should provide canTravel(), deduct() and recharge() methods. Its private balance prevents direct modification and ensures that a journey is rejected if the remaining balance would fall below Rs 50.

2. How a Fare Revision Will Be Carried Out

If the metro changes its fares, the fare constants or fare table must be updated. For example, if the Outer Zone fare changes from Rs 30 to Rs 35, the value should be updated in the central fare table.

The program should then be recompiled, tested and deployed to all ticket gates. Keeping fare values in one place avoids changing the same value in multiple parts of the program.

3. Limitations of the Design

- Fare changes require recompilation and redeployment when fares are hard-coded.
- The current design assumes fixed zones and peak-hour periods.
- New concession categories or special passes require additional rules.
- The system needs extra protection against simultaneous card transactions and network failures in a real metro environment.
- Integer-paise calculations are accurate for money, but input validation and overflow checks are still necessary.

4. Evidence from Experiments

The earlier tests support this design:

Experiment	**Result**	**What it proves**
Inner Zone, normal hour	Rs 10.00	Correct base fare

Middle Zone, peak hour	Rs 24.00	Correct 20% peak surcharge
Outer Zone, normal hour	Rs 30.00	Correct zone-based fare
Balance Rs 60, fare Rs 10	Rs 50 remaining	Minimum balance rule works
Balance Rs 59, fare Rs 10	Journey rejected	Insufficient balance is prevented
20,000 journeys	Rs 3,84,000	Integer-paise total is safe
500,000 journeys	Rs 96,00,000	32-bit integer paise total is still safe
3,000,000 journeys	Rs 5,76,00,000	Requires long long for paise totals

The experiment also showed that a typical 32-bit int cannot store 5,760,000,000 paise, whereas long long can. The double and integer-paise totals matched for the tested whole-rupee fare pattern. However, integer paise remains the safer choice when exact monetary values such as Rs 10.80 are involved.

5. Alternative Concept: Read Fares from a Configuration File

Instead of hard-coding fares, the metro can store them in a configuration file. Each gate reads the file when it starts.

Example fares.txt:

20
30

These values represent the Inner, Middle and Outer Zone fares in rupees.

Example C++ implementation:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("fares.txt");

    if (!file) {
        cout << "Error: Cannot open fare file.\n";
        return 1;
    }

    long long fares[3];

    for (int i = 0; i < 3; i++) {
        if (!(file >> fares[i]) || fares[i] < 0) {
            cout << "Error: Invalid fare configuration.\n";
            return 1;
        }

        fares[i] *= 100; // Convert rupees to paise
    }

    file.close();
cout << "Fare configuration loaded successfully.\n";
```

```
    cout << "Inner Zone: Rs " << fares[0] / 100.0 << '\n';
    cout << "Middle Zone: Rs " << fares[1] / 100.0 << '\n';
    cout << "Outer Zone: Rs " << fares[2] / 100.0 << '\n';

    return 0;
}
```

Complexity and Comparison

Let z represent the number of zones.

Factor	**Hard-coded fare table**	**Configuration file**
Start-up time	O(1)	O(z) to read and validate fares
Extra storage	O(1)	O(z) for the loaded fare table
Fare revision	Recompile and redeploy	Edit the file and reload/restart gates
Flexibility	Lower	Higher
Configuration errors	Fewer external file errors	File may be missing or contain invalid values

For a small number of zones, reading the configuration file adds very little start-up time and memory usage. It also avoids recompiling the entire program for every fare revision.

However, the metro must validate the file and ensure all gates use the same approved fare version. A real system should use controlled updates, version numbers and safe configuration reloads.

6. Final Recommendation

I recommend using an object-oriented SmartCard design with integer-paise calculations and a configuration-based fare table.

This combines the safety of private card balances, accurate monetary calculations and the flexibility of updating fares without recompiling the program. The experiments demonstrate that long long can safely handle the tested network totals, while a typical 32-bit int cannot handle the largest paise total.

Although the design handles the tested zones, peak periods, balance boundaries and large journey counts, a production metro system would also need secure card transactions, configuration validation, concurrency control and recovery mechanisms.

Conclusion: The recommended design is suitable for the whole network when all gates use the same validated fare configuration and the system includes appropriate transaction and overflow protection.